

ReactJS 관련된 링크들

W3schools.com React

<https://www.w3schools.com/react/default.asp>

React KeyCode

<https://www.npmjs.com/package/react-keyboard-event-handler>

Load your app with ?react_perf in the query string

<https://reactjs.org/blog/2016/11/16/react-v15.4.0.html>

7 Top React State Management Libraries

<https://www.trio.dev/react/resources/7-top-react-state-management-libraries>

ReactJS 19 New features

<https://medium.com/@zero86/react-react-19-%EA%B8%B0%EB%8A%A5-%EB%8B%A4%EC%8B%9C-%EC%82%B4%ED%8E%B4%EB%B3%B4%EA%B8%B0-11fe39e39b7d>

<https://medium.com/@zero86/react-%EB%A6%AC%EC%95%A1%ED%8A%B8-19-4cb65304f6f6>

js -> html dom

jquery에서는 html dom을 핸들링 할때 js에서 html dom에 부여된 id로 찾아서 object 를 처리하는 방식

react,vuejs 는 html dom을 핸들링 할때 dom 엘리먼트와 변수(state)를 연결해 놓고 변수의 값을 변경하는 방식

SPA(Single Page Application)

: index.html , html 페이지가 1개만 있는 어플리케이션

: 동적으로 html 작성하는 역할은 ReactJS가 담당함

: 링크를 작성해서 화면을 전환하려면 클라이언트를 위한 Router 가 필요하다.

- React Router

: javascript에서 html dom직접 접근 하는 방식보다는 html element와 연결된 상태변수를 선언하고

해당 상태변수를 변경하면 html element가 변경된다.

- Redux, Zustand

React <StrictMode>에 대한 질문

<https://gemini.google.com/share/3877d9b8459a>

TodoList(클래스컴포넌트) Source

<https://github.com/vega2krookies5/React-TodoList-Vite-App>

CRA는 현재는 **Deprecate** 되어 잘 사용하지 않음 주로 **Vite**를 사용함
 하지만 **Vite** 이전에 많이 사용되었던 **Build** 도구였기 때문에 설명을 하고
 있습니다.

CRA(Create React App) 에 대한 설명

<https://create-react-app.dev/>

\$ npm i -g create-react-app

CRA 의 기능

: reactjs skeleton 코드를 작성 해주는 Build Tool

CRA 버전 확인

\$ create-react-app --version

CRA가 reactjs19 지원 하지 않는 문제

<https://github.com/facebook/react/issues/32016>

CRA는 내부적으로 Webpack 과 Babel, ESLint 를 사용한다.

Webpack 과 Babel, ESLint 하는 역할은 무엇일까요?

1) Babel <https://babeljs.io/> , SWC <https://swc.rs/>

: Javascript 컴파일러 (트랜스파일러)

: ES6 -> ES5 로 변환 (변환하는 이유는 브라우저 호환성을 위해서)

: TS -> ES5 (타입스크립트의 tsc 컴파일러 사용)

2) ESLint <https://eslint.org/>

: javascript 정적인 코드 분석 도구 find and fix problems

3) Webpack <https://webpack.js.org/>

: module bundler 역할, Client을 위한 Dev Server - HMR(hot module replacement)

기능 제공

node_modules 하위에 있는데 라이브러리 js와 개발자가 작성한 여러개의 jsx 컴포넌트들을
 번들링 해서 배포해야 합니다.

FrontEnd 개발 할때 사용하는 Build Tools

Reactjs : CRA(Create React App)

Vuejs : Vue-CLI / Vue-create

Angular : Angular-CLI

CSR (Client Side Rendering) - ReactJS / VueJS

SSR (Server Side Rendering) - NextJS / NuxtJS / Remix

SEO - 검색 엔진 최적화

CRA를 기반으로 React 프로젝트 생성

```
$ npx create-react-app hello-cra-app
```

```
cd hello-cra-app
npm install react-scripts@latest
npm start
```

CRA 로 생성한 프로젝트의 package.json

package.json

```
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject"
},
```

webpack dev server : HMR(Hot Module Replacement) 기능 제공

compile & bundling & 개발서버 start (Dev Mode 개발환경)

```
> npm start
```

compile & bundling (Production Mode 운영환경)

```
> npm run build
```

unit test case 실행

```
> npm run test
```

감추어진 설정들을 꺼내기 (원래 대로 되돌리기는 않됨, git commit 명령이 실행되어 있어야 함)

```
> npm run eject
```

node_modules 폴더를 생성하도록 하려면 package.json

```
> npm install
```

```
> npm i
```

라이브러리 설치 후에도 오류가 발생하면

```
$ npm audit fix --force
```

CRA(Create-React-App) 로 작성된 ReactJS App 의 구조

<http://localhost:3000>

```

index.js
index.html ==> App.js ==> components/MyComponent.js
                                components/MyComponentFunc.js

```

```

public/
  index.html
  <div id="root"></div>

```

```

src/
  index.js
  const root = ReactDOM.createRoot(document.getElementById('root'));
  root.render(
    <React.StrictMode>
      <App />
    </React.StrictMode>
  );

```

```

App.js
function App() {
  return (
    <div className="App">
      <h1>Hello React</h1>
    </div>
  );
}

```

Vite를 기반으로 React 프로젝트 생성

<https://vitejs.dev/>

<https://github.com/vitejs/vite/tree/main/packages/create-vite>

```
$ npm create vite@latest reacthello-vite-app -- --template react
```

Vite 로 생성한 프로젝트의 package.json

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint . --ext js,jsx --report-unused-disable-directives --max-warnings 0",
  "preview": "vite preview"
},
```

개발환경 (컴파일 & 번들링 & dev server start)

```
$ npm run dev
```

운영환경에 배포 (컴파일 & 번들링)

```
$ npm run build
```

배포한 버전을 미리 실행볼 수 있는 명령

```
$ npm run preview
```

Vite 로 작성된 ReactJS App 의 구조

main.jsx

index.html ==> App.jsx ==> src/components/MyComponent.jsx

index.html

```
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

src/main.jsx

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

src/App.jsx

```
function App() {
  const [count, setCount] = useState(0)
}
```

TodoList App 호출 흐름

```
main.jsx
index.html ==> App.jsx ==> src/components/ToDoListTemplate.jsx
                                ==> Form.jsx
                                ==> TodoItem.jsx
                                    ==> TodoItem.jsx
```

함수형 컴포넌트 형태로 작성한 TodoList Repository 주소

git clone <https://github.com/vega2kgkubee/React-TodoList-Vite-App-Function>

메모이제이션 (Memoization)에 대한 질문

<https://gemini.google.com/share/3e2d711ddab7>

<https://gemini.google.com/share/745135412caa>

JS와 JSX의 차이점 및 React에서 JSX 사용 이유

1. **JS (JavaScript):**
 - 순수 JavaScript 언어
 - 표준 ECMAScript 사양을 따름
 - 브라우저에서 직접 실행 가능
 - HTML과 분리된 순수 코드
2. **JSX (JavaScript XML):**
 - JavaScript의 확장 문법
 - HTML과 유사한 템플릿을 JavaScript 코드 안에 작성할 수 있게 해줌
 - `React.createElement()` 호출로 변환됨
 - 브라우저에서 직접 실행 불가 (Babel 로 변환이 필요함)

React에서 JSX를 사용하는 이유

1. 가독성 향상:
2. 시각적 직관성:
 - a. React 컴포넌트 구조를 HTML과 유사하게 표현하여 UI 계층을 쉽게 파악 가능
3. 생산성 증가:
 - a. 복잡한 UI를 더 쉽고 빠르게 작성 가능
 - b. 개발자 경험(Developer Experience) 향상
4. React의 설계 철학:
 - a. "템플릿과 로직의 분리"가 아닌 "관심사의 분리"를 구현
 - b. 마크업과 로직을 같은 파일에 함께 두는 것이 더 자연스러움
5. 강력한 JavaScript 기능 활용:
 - a. JSX 내부에서 JavaScript 표현식을 직접 사용 가능

주의사항

- JSX는 반드시 트랜스파일러(예: Babel)를 통해 일반 JavaScript로 변환되어야 함
- 모든 JSX 태그는 닫혀야 함 (`<div></div>` 또는 `<br />`)
- 최상위 요소는 하나만 있어야 함 (React 16 이후로는 Fragment `<></>` 사용 가능)
- `class` 대신 `className`을 사용 `<button className="btn" >`
- 이벤트 핸들러 구현 `onclick` 아니라 반드시 `onClick=""`, `onKeyDown`

1. JSX를 사용한 경우

index.html

```
<div id="container">
  <!-- This element's contents will be replaced with your component. -->
</div>
```

index.js(x)

```
ReactDOM.render(
  <Hello name="World" />,
  document.getElementById('container')
);
```

Hello.js(x)

```
class Hello extends React.Component {
  render() {
    return <div>Hello {this.props.name}</div>;
  }
}
```

2. JS를 사용한 경우 (JSX를 사용하지 않은 경우)

index.html

```
<div id="container">
  <!-- This element's contents will be replaced with your component. -->
</div>
```

index.js

```
ReactDOM.render(
  React.createElement(Hello, {name: "World"}),
  document.getElementById('container')
);
```

Hello.js

```
class Hello extends React.Component {
  render() {
    return React.createElement("div", null, "Hello ", this.props.name);
  }
}
```

React 컴포넌트의 2가지 유형

1. 클래스 타입 컴포넌트 (extends Component)

```
class App extends Component {
  const name = 'Hello';
  //overriding(재정의)
  render() {
    return ( <div>{name}</div> )
  }
};
```

: state 객체(상태 변수) 선언 할 수 있고, **LifeCycle** 메서드 재정의(렌더링 성능 최적화) 할 수 있다.

2. 함수형 컴포넌트 (주로 많이 사용되는 방식)

```
function App() {
  return ( <div></div> )
}
```

: 렌더링이 클래스 컴포넌트 보다 빠르다.

: 제약사항 - state 객체(상태변수) 선언 할 수 없고, **LifeCycle** 메서드 재정의 할 수 없다.

React Hooks 함수 제공(ver 16.8)

: 함수형 컴포넌트의 제약사항들을 **Support** 해주는 역할

: **useState()** 함수를 사용하여 상태 변수를 선언할 수 있다.

: **useMemo(), useEffect(), useCallback()**를 Hook 함수를 사용하면 렌더링 성능 최적화를 할 수 있다.

Environment 별로 환경설정 파일 작성하기

Create-React-App 를 사용했을 경우

npm start 할 때의 Mode는 development
 npm run build 할 때의 Mode는 production
 .env.{mode}

Development 환경

=> .env.development

REACT_APP_APIURL=<http://localhost:4500/api/todos>

REACT_APP_MODE=Dev

Production 환경

=> .env.production

REACT_APP_APIURL=<http://domain/api/todos>

REACT_APP_MODE=Prod

환경변수를 사용하기

process.env.REACT_APP_APIURL

Deployment에 대한 내용

<https://create-react-app.dev/docs/deployment>

custom staging mode 추가하는 방법

<https://create-react-app.dev/docs/deployment/#customizing-environment-variables-for-arbitrary-build-environments>

VITE 를 사용했을 경우

.env.development

VITE_API_URL=<http://localhost:4500/api/todos>

VITE_MODE=Dev

환경변수를 사용하기

import.meta.env.VITE_API_URL;

개발 환경 - compile, bundling, dev server start

.env.development 환경설정이 적용됨

> npm run dev

운영환경 - compile, bundling, dist/ 폴더에 번들링한 js,css 파일이 만들어짐

.env.production 환경설정이 적용됨

> npm run build

빌드된 파일을 배포하기 전에 테스트를 위한 명령

> npm run preview

VITE 를 사용한 프로젝트에서 절대경로 설정하기

[vite.config.js](#)

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: [
      { find: "@", replacement: "/src" },
      { find: "@components", replacement: "/src/components" },
    ]
  }
})
```

Redux의 구성요소

1. Store : 저장소
 - : 상태정보 (state 객체)를 저장하는 저장소 역할을 하는 객체
 - : createStore(myReducer) 함수를 호출해서 store 객체를 생성 할 수 있음
2. Action: 요청
 - : 상태 정보를 변경하는 요청 정보가 포함된 함수
 - : {type:ADD, payload: todolist} 객체를 반환하는 Action 함수
3. Dispatch : Action을 전달함
 - : Action(요청)을 Store에게 전달하는 함수
 - : store.dispatch(action)
4. Reducer 상태변수 변경
 - : Action(요청) type에 따라서 상태정보(state객체)의 값을 변경하는 pure function 순수 함수
 - : function myReducer(state, action) { }
5. Subscribe : 변경된 정보를 전달 받는 함수

subscribe 함수는 react-redux 라이브러리에서 제공하는

connect(mapStateToProps, mapDispatchToProps) 함수로 대체 되어진다.

Redux Cartoons

<https://bestalign.github.io/translation/cartoon-intro-to-redux/>

Redux 만 사용한 Counter 예제

```
const INCREMENT = 'INCREMENT';
const DECREMENT = 'DECREMENT';
//값을 증가시키는 액션
const increment = () => ({
  type : INCREMENT
});
//가변적인 값이 들어가야 하면 파라미터를 넣어서 액션을 만든다.
const increment2 = (diff) => ({
  type : INCREMENT,
  diff : diff
});
//값을 감소시키는 액션
const decrement = (diff) => ({
  type : DECREMENT,
  diff : diff
});
-----
//state객체
const initialState = { number : 0,foo:'bar',baz:'qux' }
//Reducer 함수
function counter(state = initialState, action) {
  switch(action.type) {
    case INCREMENT:
      return {
        ...state,
        number:state.number + action.diff
      };
    case DECREMENT:
      return {
        ...state,
```

```

    number: state.number - action.diff
  };
  default:
    return state;
}
}

```

```

const { createStore } = Redux;

```

```

//store 객체 생성
const store = createStore(counter);
store.subscribe(() => {
  console.log('State 객체 확인')
  const state = store.getState();
  console.log(state);
  const {number, baz, foo} = state;
  console.log(number);
  console.log(baz);
  console.log(foo);
});

```

```

//dispatch 함수를 호출해서 action을 전달하기
store.dispatch(increment2(5));
store.dispatch(decrement(3));

```

index.js
 index.html ==> src/containers/App.js ==> [Counter.js](#)

index.js
 index.html ==> src/containers/App.js ==> CounterContainer.js ==> Counter.js
 connect(mapStateToProps, mapDispatchToProps)

1. CounterContainer 가 Store객체와 연결되어 있는 컴포넌트
2. Counter 컴포넌트의 부모는 CounterContainer 이다.
 - 1) connect() 두번째 인자인 mapDispatchToProps 함수
 mapDispatchToProps 에서 dispatch 함수를 호출하면서 아규먼트로 액션함수를 전달합니다.
 dispatch() 함수의 Property 를 Counter 에게 props 로 전달해 준다.

<script type="module"> 설명

`type="module"` 속성은 자바스크립트 파일을 **ES6 모듈(ECMAScript 모듈)** 방식으로 실행하도록 지정하는 속성입니다.

이 속성을 사용하면 모든 자바스크립트의 모듈 시스템을 활용할 수 있습니다.

주요 특징

1. 모듈 스코프 적용
 - 모듈 내부의 변수/함수는 기본적으로 `private` 스코프를 가집니다.
 - `export`로 명시적으로 내보내지 않은 내용은 외부에서 접근할 수 없습니다.
2. 엄격 모드(**Strict Mode**) 자동 적용
 - 모듈 내의 코드는 자동으로 `strict mode`로 실행됩니다.
3. 지연 실행(**defer**)
 - 모듈 스크립트는 기본적으로 HTML 파싱을 방해하지 않으며, 문서 로딩이 완료된 후 실행됩니다.
4. **CORS** 요구
 - 모듈 스크립트는 동일 출처 정책(Same-origin policy)을 따르며, CORS 헤더가 필요합니다.

```
<script type="module" src="/src/main.js"></script>
```

일반 스크립트와의 차이점

특징	일반 스크립트 (<script>)	모듈 스크립트 (<script type="module">)
스코프	전역 스코프	모듈 스코프 (로컬)
엄격 모드	명시적으로 지정 필요	자동 적용
실행 시점	기본적으로 즉시 실행	defer 속성 자동 적용
import/export 문법	사용 불가	사용 가능
CORS 요구사항	없음	필수

엄격 모드(Strict Mode) 설명

엄격 모드는 JavaScript에서 더 엄격한 문법 검사와 오류 처리를 활성화하는 모드입니다.

ES5(ECMAScript 5)에서 처음 도입되었으며, 코드의 안정성을 높이기 위해 설계되었습니다.

엄격 모드의 주요 특징

1. 실수 방지

: 암묵적 전역 변수 생성 금지 (선언 없이 변수 사용시 오류)

```
'use strict';
```

```
x = 10; // ReferenceError: x is not defined
```

2. 읽기 전용 속성/함수 수정 금지

```
'use strict';
```

```
NaN = 1; // TypeError: Cannot assign to read only property 'NaN'
```

3. 예약어 보호

: 미래의 ECMAScript 버전을 위한 키워드(e.g., `interface`, `let`, `package`)를 변수명으로 사용 금지

NodeJs 기반 Express 로 작성한 Todo REST API 호출하기

1. Todo 목록

GET

<http://localhost:4500/api/todos>

2. Todo 를 id로 조회

GET

<http://localhost:4500/api/todos/1>

3. Todo 등록

POST

<http://localhost:4500/api/todos>

request header

content-type: application/json

request body : raw

```
{  
  "text": "Todo4",  
  "checked": true  
}
```

4. Todo 수정

PUT vs PATCH

- put 전체 항목 수정
- patch 부분 항목 수정

PATCH

<http://localhost:4500/api/todos/1>

request header

content-type: application/json

request body : raw

```
{ "checked": true }
```

5. Todo 삭제

DELETE

<http://localhost:4500/api/todos/4>